Week 7: MEC6602E Transonic Aerodynamics

Frédéric Plante, ing., Ph.D.

Polytechnique Montréal

October 20, 2026



This week

- Second-order Roe scheme
- Acceleration techniques:
  - Local time stepping;
  - Implicit residual smoothing;
  - Enthalpy damping;
  - Implicit scheme;
  - Multigrid method.
- Parallelization

References

Algorithms shown in this presentation (figures and equations) are taken from:

- Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.
- Cagnone, J. S., Sermeus, K., Nadarajah, S. K., & Laurendeau, E. (2011). Implicit multigrid schemes for challenging aerodynamic simulations on block-structured grids. *Computers & Fluids*, 44(1), 314-327.

The Blazek book refers to the original articles.

Second-order Roe scheme

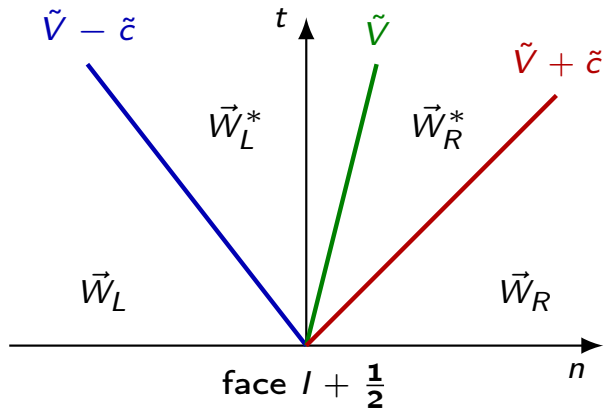
Roe scheme

At each face, we have a Riemann problem between $\vec{W}_L$ and $\vec{W}_R$. Roe solves a linearized problem:

$$\frac{\partial \vec{W}}{\partial t} + \bar{A}_{Roe} \frac{\partial \vec{W}}{\partial n} = 0$$

The matrix $\bar{A}_{Roe} = \bar{A}(\vec{W}_L, \vec{W}_R)$ must satisfy:

1. $\vec{F}_c(\vec{W}_R) - \vec{F}_c(\vec{W}_L) = \bar{A}_{Roe}\,(\vec{W}_R - \vec{W}_L)$;
2. $\bar{A}_{Roe}(\vec{W}, \vec{W}) = A(\vec{W})$;
3. Real eigenvalues and a complete set of eigenvectors.



Subsonic case, $\tilde{V} > 0$

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.
Roe, P. L. (1981). Approximate Riemann solvers, parameter vectors, and difference schemes. *Journal of Computational Physics*, 43(2), 357-372.

Roe scheme

Scalar case: linear advection $\frac{\partial u}{\partial t} + a\frac{\partial u}{\partial x} = 0,\ f = a\,u$

$$f_{l+1/2} = \underbrace{\tfrac{1}{2}\left(f_L + f_R\right)}_{\text{central}} - \underbrace{\tfrac{1}{2}|a|\left(u_R - u_L\right)}_{\text{dissipation}}$$

$a > 0: \quad f_{l+1/2} = a\,u_L$

information comes from the left

$a < 0: \quad f_{l+1/2} = a\,u_R$

information comes from the right

With $|a|$, the scheme is upwind whatever the sign of a.

For the Euler equations, $|\bar{A}_{Roe}|$ does the same thing for each wave ($\tilde{V} - \tilde{c}$, $\tilde{V}$, $\tilde{V} + \tilde{c}$).

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.
Roe, P. L. (1981). Approximate Riemann solvers, parameter vectors, and difference schemes. *Journal of Computational Physics*, 43(2), 357-372.



Roe matrix

We decompose the Roe matrix with its eigenvectors:

$$\bar{A}_{Roe} = T\,\Lambda\,T^{-1}, \qquad |\bar{A}_{Roe}| = T\,|\Lambda|\,T^{-1}, \qquad |\Lambda| = \mathrm{diag}\big(|\lambda_1|,\ldots,|\lambda_5|\big)$$

The jump at the face is decomposed into waves $\vec{W}_R - \vec{W}_L = \sum_k \alpha_k \vec{r}_k$:

$$|\bar{A}_{Roe}|\,(\vec{W}_R - \vec{W}_L) = \sum_{k=1}^{5} |\lambda_k|\,\alpha_k\,\vec{r}_k = |\Delta\vec{F}_1| + |\Delta\vec{F}_{2,3,4}| + |\Delta\vec{F}_5|$$

k	speed λ_k	wave	strength α_k
1	$\tilde{V} - \tilde{c}$	acoustic	$(\Delta p - \tilde{\rho}\tilde{c}\,\Delta V)/(2\tilde{c}^2)$
2	$\tilde{V}$	entropy	$\Delta\rho - \Delta p/\tilde{c}^2$
3, 4	$\tilde{V}$	shear (tangential velocity)	$\tilde{\rho}\,(\Delta\vec{v} - \Delta V\,\vec{n})$
5	$\tilde{V} + \tilde{c}$	acoustic	$(\Delta p + \tilde{\rho}\tilde{c}\,\Delta V)/(2\tilde{c}^2)$

$\vec{r}_k$: right eigenvectors (columns of T). We do not need to compute T^{-1}.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Roe, P. L. (1981). Approximate Riemann solvers, parameter vectors, and difference schemes. *Journal of Computational Physics*, 43(2), 357-372.

Flux-difference splitting scheme: Roe scheme

$$(\vec{F}_c)_{I+1/2} = \frac{1}{2}\left[\vec{F}_c(\vec{W}_R) + \vec{F}_c(\vec{W}_L) - |\bar{A}_{Roe}|_{I+1/2}\,(\vec{W}_R - \vec{W}_L)\right]$$

$$|\bar{A}_{Roe}|(\vec{W}_R - \vec{W}_L) = |\Delta\vec{F}_1| + |\Delta\vec{F}_{2,3,4}| + |\Delta\vec{F}_5|$$

$$|\Delta\vec{F}_{\mathbf{1}}| = |\tilde{V} - \tilde{c}|\left(\frac{\Delta p - \tilde{\rho}\tilde{c}\Delta V}{2\tilde{c}^{\mathbf{2}}}\right)\begin{bmatrix}1\\ \tilde{u} - \tilde{c}n_x\\ \tilde{v} - \tilde{c}n_y\\ \tilde{w} - \tilde{c}n_z\\ \tilde{H} - \tilde{c}\tilde{V}\end{bmatrix}$$

$$|\Delta\vec{F}_{\mathbf{5}}| = |\tilde{V} + \tilde{c}|\left(\frac{\Delta p + \tilde{\rho}\tilde{c}\Delta V}{2\tilde{c}^{\mathbf{2}}}\right)\begin{bmatrix}1\\ \tilde{u} + \tilde{c}n_x\\ \tilde{v} + \tilde{c}n_y\\ \tilde{w} + \tilde{c}n_z\\ \tilde{H} + \tilde{c}\tilde{V}\end{bmatrix}$$

$$|\Delta\vec{F}_{\mathbf{2,3,4}}| = |\tilde{V}|\left\{\left(\Delta\rho - \frac{\Delta p}{\tilde{c}^{\mathbf{2}}}\right)\begin{bmatrix}1\\ \tilde{u}\\ \tilde{v}\\ \tilde{w}\\ \tilde{q}^{\mathbf{2}}/2\end{bmatrix} + \tilde{\rho}\begin{bmatrix}0\\ \Delta u - \Delta V n_x\\ \Delta v - \Delta V n_y\\ \Delta w - \Delta V n_z\\ \tilde{u}\Delta u + \tilde{v}\Delta v + \tilde{w}\Delta w - \tilde{V}\Delta V\end{bmatrix}\right\}$$

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Flux-difference splitting scheme: Roe scheme

˜ means the Roe average state:

$$\tilde{\rho} = \sqrt{\rho_L \rho_R}$$

$$\tilde{u} = \frac{u_L\sqrt{\rho_L} + u_R\sqrt{\rho_R}}{\sqrt{\rho_L} + \sqrt{\rho_R}}$$

$$\tilde{v} = \frac{v_L\sqrt{\rho_L} + v_R\sqrt{\rho_R}}{\sqrt{\rho_L} + \sqrt{\rho_R}}$$

$$\tilde{w} = \frac{w_L\sqrt{\rho_L} + w_R\sqrt{\rho_R}}{\sqrt{\rho_L} + \sqrt{\rho_R}}$$

$$\tilde{H} = \frac{H_L\sqrt{\rho_L} + H_R\sqrt{\rho_R}}{\sqrt{\rho_L} + \sqrt{\rho_R}}$$

$$\tilde{c} = \sqrt{(\gamma - 1)\left(\tilde{H} - \tilde{q}^2/2\right)}$$

$$\tilde{V} = \tilde{u}n_x + \tilde{v}n_y + \tilde{w}n_z$$

$$\tilde{q}^2 = \tilde{u}^2 + \tilde{v}^2 + \tilde{w}^2$$

Δ means the difference between the state at the right and at the left of the face: $\Delta(\cdot) = (\cdot)_R - (\cdot)_L$.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Roe average

The Roe average is the state for which the first property is exact:

$$\vec{F}_c(\vec{W}_R) - \vec{F}_c(\vec{W}_L) = A(\tilde{\vec{W}})\,(\vec{W}_R - \vec{W}_L)$$

- For a stationary shock, $\vec{F}_R = \vec{F}_L$ and $|\bar{A}_{Roe}|\Delta\vec{W} = 0$. The shock is captured without added dissipation;
- When an eigenvalue goes to zero (sonic point), there is no dissipation for this wave. We can get a non-physical expansion shock. This is why we need an entropy correction.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.
Roe, P. L. (1981). Approximate Riemann solvers, parameter vectors, and difference schemes. *Journal of Computational Physics*, 43(2), 357-372.

Roe scheme: entropy correction

This scheme requires an entropy correction. The Harten entropy correction is:

$$|\Lambda_c| = \begin{cases} |\Lambda_c| & \text{if } |\Lambda_c| > \delta \\ \dfrac{\Lambda_c^2 + \delta^2}{2\delta} & \text{if } |\Lambda_c| \leq \delta \end{cases}$$

applied to each eigenvalue: $|\Lambda_c| = |\tilde{V} \pm \tilde{c}|$ and $|\tilde{V}|$.

δ is set to a small fraction of the local speed of sound (like $\tilde{c}/10$).

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Roe scheme implementation

For each face:

1. Compute $\vec{W}_L$ and $\vec{W}_R$ (cell values at first order, MUSCL at second order);
2. Compute the Roe average state $\tilde{\rho}$, $\tilde{u}$, $\tilde{v}$, $\tilde{H}$, $\tilde{c}$ and $\tilde{V}$;
3. Compute the eigenvalues with the entropy correction;
4. Compute $|\Delta\vec{F}_1|$, $|\Delta\vec{F}_{2,3,4}|$ and $|\Delta\vec{F}_5|$;
5. Compute the flux and multiply by the face length ΔS.

To verify: with a uniform flow, the residual must be zero.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Second-order scheme

Monotone Upstream-centred Schemes for Conservation Laws (MUSCL) for a structured mesh:

$$U_R = U_{I+1} - \frac{\epsilon}{4}\left[(1+\hat{\kappa})\Delta_- + (1-\hat{\kappa})\Delta_+\right]U_{I+1}$$

$$U_L = U_I \quad + \frac{\epsilon}{4}\left[(1+\hat{\kappa})\Delta_+ + (1-\hat{\kappa})\Delta_-\right]U_I$$

$$\Delta_+ U_I = U_{I+1} - U_I, \qquad \Delta_- U_I = U_I - U_{I-1}$$

- $\epsilon = 0$: first-order accurate upwind discretization;
- $\epsilon = 1$, $\hat{\kappa} = -1$: fully one-sided interpolation, second-order upwind approximation;
- $\epsilon = 1$, $\hat{\kappa} = 0$: second-order upwind-biased linear interpolation;
- $\epsilon = 1$, $\hat{\kappa} = 1/3$: three-point interpolation formula.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

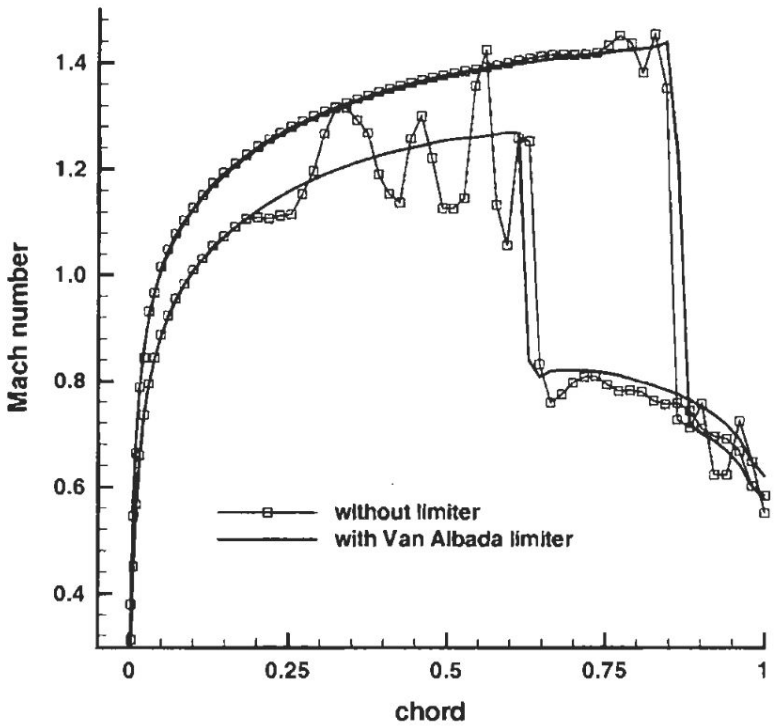
Second-order scheme

We introduce a limiter to maintain the monotonicity of the solution:

- Maxima must be non-increasing;
- Minima must be non-decreasing;
- No local extrema created

in the time evolution.

Near a discontinuity, the limiter must reduce the order of the scheme to first order.



Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Second-order Roe scheme

$$U_R = U_{I+1} - \frac{1}{4}\left[(1+\hat{\kappa})\Phi^{+}_{I+1/2}\Delta_{-} + (1-\hat{\kappa})\Phi^{-}_{I+3/2}\Delta_{+}\right]U_{I+1}$$

$$U_L = U_I \quad + \frac{1}{4}\left[(1+\hat{\kappa})\Phi^{-}_{I+1/2}\Delta_{+} + (1-\hat{\kappa})\Phi^{+}_{I-1/2}\Delta_{-}\right]U_I$$

We skip the derivation (shown in Blazek):

$$U_R = U_{I+1} - \frac{1}{2}\Psi_R\left(U_{I+2} - U_{I+1}\right)$$

$$U_L = U_I \quad + \frac{1}{2}\Psi_L\left(U_I - U_{I-1}\right)$$

$$\Psi_{L/R} = \frac{1}{2}\left[(1+\hat{\kappa})\, r_{L/R} + (1-\hat{\kappa})\right]\Phi_{L/R}$$

$$r_R = \frac{U_{I+1} - U_I}{U_{I+2} - U_{I+1}} = \frac{\Delta_{-}}{\Delta_{+}}U_{I+1}$$

$$r_L = \frac{U_{I+1} - U_I}{U_I - U_{I-1}} = \frac{\Delta_{+}}{\Delta_{-}}U_I$$

We need to choose a proper limiter Φ.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Second-order scheme: limiters

If $\hat{\kappa} = 0$, we can take the Van Albada limiter:

$$\Phi(r) = \frac{2r}{r^2+1}, \qquad \Psi(r) = \frac{r^2+r}{1+r^2}$$

$$\begin{aligned} U_R &= U_{I+1} - \tfrac{1}{2}\delta_R \\ U_L &= U_I \quad + \tfrac{1}{2}\delta_L \end{aligned}$$

$$\delta = \frac{a(b^2+\epsilon) + b(a^2+\epsilon)}{a^2+b^2+2\epsilon}$$

$$\begin{aligned} a_R &= \Delta_+ U_{I+1}, \quad & b_R &= \Delta_- U_{I+1} \\ a_L &= \Delta_+ U_I, & b_L &= \Delta_- U_I \end{aligned}$$

If $\hat{\kappa} = 1/3$, we can take the Hemker and Koren limiter:

$$\Phi(r) = \frac{3r}{2r^2 - r + 2}$$

$$\delta = \frac{(2a^2+\epsilon)b + (b^2+2\epsilon)a}{2a^2 + 2b^2 - ab + 3\epsilon}$$

ϵ is a small number. We can use the square of the cell volume.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

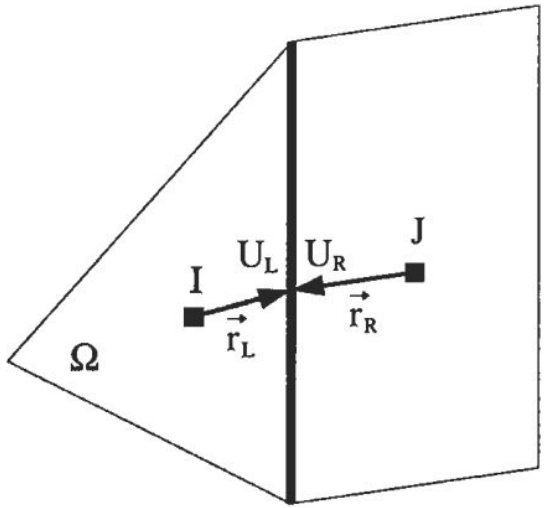
Unstructured mesh implementation

Not used in this course

Piecewise linear reconstruction:

$$U_L = U_I + \Psi_I \left(\nabla U_I \cdot \vec{r_L}\right)$$
$$U_R = U_J + \Psi_J \left(\nabla U_J \cdot \vec{r_R}\right)$$

We use the gradient (computed with the Green-Gauss or the weighted least-squares approach) to reconstruct the solution at the face, and we add a limiter.



Accuracy is linked to the dissipation introduced by the limiter function. Some limiters are less dissipative.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

JST scheme for unstructured meshes

Not used in this course

Central scheme with artificial dissipation (JST). To implement the JST scheme on an unstructured mesh, we use the Laplacian (divergence of the gradient) for the second-order term and the Laplacian of the Laplacian for the fourth-order term.

$$\vec{D}_I = \sum_{J=1}^{N_A} (\hat{\Lambda}_c)_{IJ}\, \epsilon_{IJ}^{(2)}\, \theta_{IJ} \left(\vec{W}_J - \vec{W}_I \right) - \sum_{J=1}^{N_A} (\hat{\Lambda}_c)_{IJ}\, \epsilon_{IJ}^{(4)}\, \theta_{IJ} \left[L(\vec{W}_J) - L(\vec{W}_I) \right]$$

$$L(U_I) = \sum_{J=1}^{N_A} \theta_{IJ} \left(U_J - U_I \right)$$

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Acceleration techniques

Local time stepping

Can be used in Homework 2

For steady-state simulations, it is possible to use a time step based on the CFL condition of each cell. The largest time step allowed by a given CFL number is used in each cell:

$$\Delta t_I = \sigma \frac{\Omega_I}{\left(\hat{\Lambda}_c^I + \hat{\Lambda}_c^J\right)_I}, \qquad \hat{\Lambda}_c^I = \left(|\vec{V} \cdot \vec{n}^I| + c\right)\Delta S^I, \quad \hat{\Lambda}_c^J = \left(|\vec{V} \cdot \vec{n}^J| + c\right)\Delta S^J$$

- Allow each cell to evolve at its own largest time step;
- Sacrifice temporal accuracy (not required for steady state) for speed;
- Almost always used.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Implicit residual smoothing

Can be used in Homework 2

To improve convergence, it is possible to add an "implicit" character by smoothing the residuals. The update in one cell depends on its neighbours. It also helps to damp the high-frequency errors. On an unstructured grid, this can be done by:

$$\underbrace{\bar{R}_I + \sum_{J=1}^{NF} \epsilon \left(\bar{R}_I - \bar{R}_J\right)}_{\text{smoothed residuals}} = \underbrace{R_I}_{\text{residual from the scheme}}$$

The sum of the coefficients must be zero.

This results in a large matrix (all the residuals are coupled to those of the neighbour cells). Solving it is costly for large problems. The solution is to use a Jacobi iteration:

$$\bar{R}_I^{(k+1)} = \frac{R_I + \sum_{J=1}^{NF} \epsilon \, \bar{R}_J^{(k)}}{1 + NF\,\epsilon}, \qquad \bar{R}^{(0)} = R, \qquad 0.5 \leq \epsilon \leq 0.8$$

Two iterations should be sufficient (we do not need to fully converge this system). This normally allows us to double the CFL number for explicit time integration.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Jacobi method

To solve $A\,x = b$, with

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}$$

while convergence not reached **do**
  for $i = 1, 2, \ldots, n$ **do**

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij}\, x_j^{(k)} \right)$$

$k \leftarrow k + 1$

For the residual smoothing:
$x^{(0)} = R$ and $b = R$.

- We can use a convergence criterion based on the residual, but in practice the algorithm converges in a few iterations. We can fix the number of iterations at 2.
- A sufficient condition for the Jacobi iteration to converge is that the matrix is diagonally dominant: $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$. This is the case by construction in this case.

Implicit residual smoothing

Can be used in Homework 2

On a structured grid, this can be done (in 3D) by:

$$-\epsilon^I \vec{R}^*_{I-1,J,K} + (1+2\epsilon^I)\vec{R}^*_{I,J,K} - \epsilon^I \vec{R}^*_{I+1,J,K} = \vec{R}_{I,J,K}$$

$$-\epsilon^J \vec{R}^{**}_{I,J-1,K} + (1+2\epsilon^J)\vec{R}^{**}_{I,J,K} - \epsilon^J \vec{R}^{**}_{I,J+1,K} = \vec{R}^*_{I,J,K}$$

$$\cancel{-\epsilon^K \vec{R}^{***}_{I,J,K-1} + (1+2\epsilon^K)\vec{R}^{***}_{I,J,K} - \epsilon^K \vec{R}^{***}_{I,J,K+1} = \vec{R}^{**}_{I,J,K}} \quad \text{3D}$$

$$\epsilon^I = \max\left\{\frac{1}{4}\left[\left(\frac{\sigma^*}{\sigma}\frac{1}{1+\Psi r}\right)^2 - 1\right], 0\right\}$$

$$\epsilon^J = \max\left\{\frac{1}{4}\left[\left(\frac{\sigma^*}{\sigma}\frac{1}{1+\Psi/r}\right)^2 - 1\right], 0\right\}$$

$$r = \hat{\Lambda}^J_c/\hat{\Lambda}^I_c, \quad \Psi \approx 0.125, \quad \frac{\sigma^*}{\sigma} \leq \sqrt{1+4\epsilon}, \quad \sigma^*/\sigma \approx 2$$

This can efficiently be solved with the Thomas algorithm (no need to have a block version, the residuals of each equation are smoothed separately).

We can double the CFL of explicit schemes with IRS method.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Enthalpy damping

Can be used in Homework 2

In an adiabatic application (no heat added or removed from the system), the enthalpy is constant. This is the case for the external flow around an airfoil solved with the Euler equations (not the Navier-Stokes equations). We can take advantage of this by adding a forcing term related to the difference between the enthalpy and its farfield value. This is done as follows:

$$\vec{W}_I^{n+1} = \frac{1}{1 + \vartheta\left(H_I^{(m)} - H_\infty\right)} \vec{W}_I^{(m)}$$

Except for the energy:

$$(\rho E)_I^{n+1} = \frac{1}{1 + \vartheta\left(H_I^{(m)} - H_\infty\right)} \left[(\rho E)_I^{(m)} - \vartheta\left(H_I^{(m)} - H_\infty\right) p_I^{(m)}\right]$$

ϑ is a constant.
This is not a very popular method because of it's limitations.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Implicit Euler scheme

Can be used in Homework 2 but challenging

We want to solve

$$\Omega_i \frac{\partial W_i}{\partial t} + R_i = 0$$

If we take R at $n+1$, the scheme is implicit:

$$\Omega_i \frac{W_i^{n+1} - W_i^n}{\Delta t} + R_i^{n+1} = \Omega_i \frac{\Delta W_i}{\Delta t} + R_i^{n+1} = 0$$

$$R_i^{n+1} = R_i + \frac{\partial R_i}{\partial W} \Delta W + O(\Delta W^2)$$

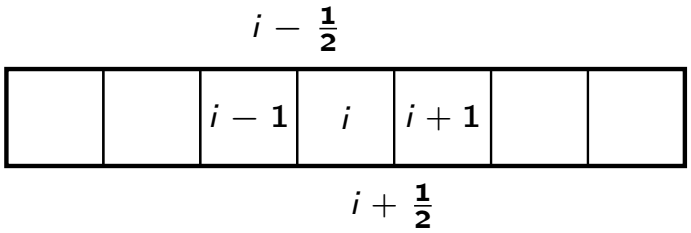
$$\Omega_i \frac{\Delta W_i}{\Delta t} + R_i + \frac{\partial R_i}{\partial W} \Delta W = 0$$

$$\boxed{\left(I \frac{\Omega_i}{\Delta t} + \frac{\partial R_i}{\partial W} \right) \Delta W = -R_i}$$

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Implicit operator derivation

How to compute the matrix? In 1D:



For simplicity, we drop the multiplication by the integrating surface in the notation.
$|\bar{A}_c| = \bar{T}\,|\bar{\Lambda}_c\,\Delta S|\,\bar{T}^{-1}$
$\bar{T}$: matrix of right eigenvectors of the Jacobian matrix
$\bar{T}^{-1}$: matrix of left eigenvectors of the Jacobian matrix
$\bar{\Lambda}_c$: diagonal matrix of the Jacobian matrix eigenvalues

A first-order upwind scheme is:

$$
\begin{aligned}
R_i &= F_{i+1/2} - F_{i-1/2} \\
&= \left[\frac{F(W_i)+F(W_{i+1})}{2} - \frac{1}{2}|A|_{i+1/2}\,(W_{i+1}-W_i)\right] \\
&\quad - \left[\frac{F(W_{i-1})+F(W_i)}{2} - \frac{1}{2}|A|_{i-1/2}\,(W_i-W_{i-1})\right]
\end{aligned}
$$

To simplify, we assume that the matrices $|A|$ do not depend on W.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Implicit operator derivation

$$\frac{\partial R_i}{\partial W_i} = \frac{1}{2}\frac{\partial F(W_i)}{\partial W_i} - \frac{1}{2}\frac{\partial F(W_i)}{\partial W_i} + \frac{1}{2}|A|_{i-1/2} + \frac{1}{2}|A|_{i+1/2} = \frac{1}{2}|A|_{i-1/2} + \frac{1}{2}|A|_{i+1/2}$$

$$\frac{\partial R_i}{\partial W_{i-1}} = -\frac{1}{2}\underbrace{\frac{\partial F(W_{i-1})}{\partial W_{i-1}}}_{A_{i-1}} - \frac{1}{2}|A|_{i-1/2}$$

$$\frac{\partial R_i}{\partial W_{i+1}} = \frac{1}{2}\underbrace{\frac{\partial F(W_{i+1})}{\partial W_{i+1}}}_{A_{i+1}} - \frac{1}{2}|A|_{i+1/2}$$

In 2D, one must also differentiate the fluxes in the J direction. This adds terms on the diagonal and two off-diagonal terms.

Using this first-order upwind scheme to approximate the implicit operator usually works. Using the exact scheme is better for convergence and stability, but this can make the implicit operator "stiffer" (less diagonally dominant) and thus harder to invert.

We can use an implicit operator based on a scheme different from the one for the residual. However, this will reduce the stability.
Since we target a steady-state solution, this does not impact the accuracy.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Implicit scheme

$$\left(I\frac{\Omega}{\Delta t}+\frac{\partial R}{\partial W}\right)\Delta W=-R$$

A small Δt helps diagonal dominance.

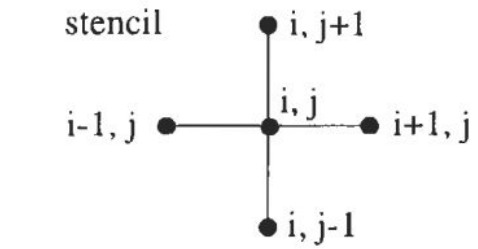
4×4 blocks in 2D

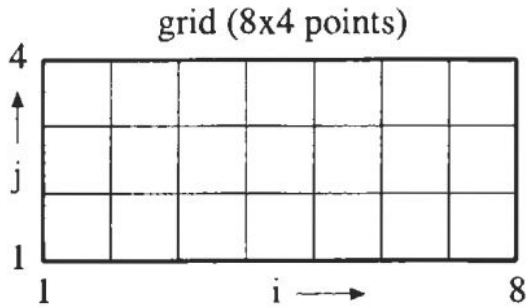
$$\underbrace{\begin{bmatrix}
\left[I\frac{\Omega_1}{\Delta t_1}+\frac{\partial R_1}{\partial W_1}\right] & \left[\frac{\partial R_1}{\partial W_2}\right] & 0 & \cdots & \cdots & 0\\
\left[\frac{\partial R_2}{\partial W_1}\right] & \left[I\frac{\Omega_2}{\Delta t_2}+\frac{\partial R_2}{\partial W_2}\right] & \left[\frac{\partial R_2}{\partial W_3}\right] & \cdots & \cdots & 0\\
\vdots & \ddots & \ddots & \ddots & & \vdots\\
0 & \cdots & \left[\frac{\partial R_i}{\partial W_{i-1}}\right] & \left[I\frac{\Omega_i}{\Delta t_i}+\frac{\partial R_i}{\partial W_i}\right] & \left[\frac{\partial R_i}{\partial W_{i+1}}\right] & 0\\
\vdots & & \ddots & \ddots & \ddots & \vdots
\end{bmatrix}}_{\left(I\frac{\Omega}{\Delta t}+\frac{\partial R}{\partial W}\right)}$$

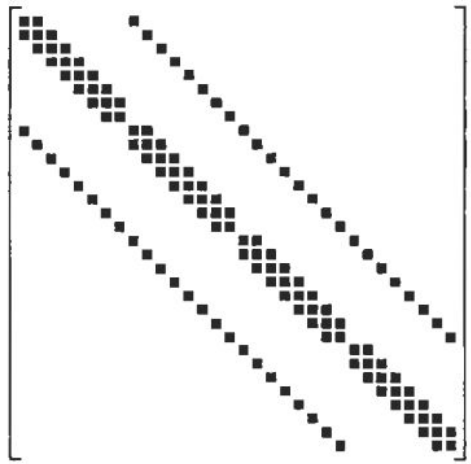
Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Implicit scheme

The structure of the implicit operator for a structured grid is like this (in 2D):





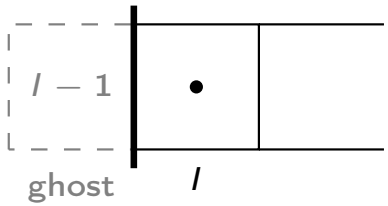


4×4 blocks. Main diagonal band: N_J series of N_I blocks on the tridiagonal ($I \pm 1$ connections). Outer bands: $J \pm 1$ connections, separated by N_I from the diagonal. This is no longer block tridiagonal.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Implicit boundary conditions

Treating the boundary conditions implicitly is quite challenging. The value in the ghost cell is not solved for, but it changes depending on the value inside the domain. So we can compute the derivative of the ghost cell value with respect to the interior cell and, by applying the chain rule, lump the coefficient on the diagonal of the implicit operator.



$$\underbrace{\frac{\partial R_I}{\partial W_{I-1}}}_{\text{flux discretization}} \Delta W_{I-1} = \frac{\partial R_I}{\partial W_{I-1}} \underbrace{\frac{\partial W_{I-1}}{\partial W_I}}_{\text{derivative of the BC}} \Delta W_I$$

This can be neglected. However, this limits the accuracy of the implicit operator and thus the convergence rate and the stability.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Shape of the implicit operator

- In 1D, the implicit operator is block tridiagonal with blocks of 3×3 matrices.
- In 2D structured, it is block tridiagonal plus 2 off-diagonal blocks of 4×4 matrices.
- In 3D structured, it is block tridiagonal plus 4 off-diagonal blocks of 5×5 matrices.
- For unstructured grids, it has a variable number of off-diagonal blocks, and the numbering can be done optimally to reduce the bandwidth of the implicit operator.

In theory the Euler implicit scheme is unconditionally stable, but in practice we must keep a finite value of CFL if the matrix is approximated or the linear system is not correctly solved.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

How to solve the implicit system?

Can be used in Homework 2 but challenging

The system resulting from the implicit scheme is costly to solve:

- Using an LU decomposition (direct inversion) is possible for 2D problems, but slow and memory consuming (prohibitive in 3D) - Practicaly not used;
- We can use an Alternating Direction Implicit (ADI) scheme to apply the implicit operator in each topological direction (IJK) separately. This reduces the problem to block tridiagonal systems (which can be solved with the Thomas algorithm) - Older method for structured grid;
- Iterative methods like Gauss-Seidel (block SGS or LU-SGS) or Jacobi iteration. Note: usually not strong enough to converge the system - Very popular method;
- Iterative Krylov methods (GMRES algorithm from a library like PETSc or other). Note: usually strong enough to converge the system - State of the art.

If you attempt an implicit solver: I recommend Gauss-Seidel.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Block Gauss-Seidel or Jacobi iteration

$$\left(I \frac{\Omega_i}{\Delta t_i} + \frac{\partial R_i}{\partial W_i} \right) \Delta W_i = -R_i - \sum_{j \neq i} \frac{\partial R_i}{\partial W_j} \Delta W_j^*$$

Gauss-Seidel iteration is a classic iterative method to solve a linear system of equations. We perform this iteration for each i in sequence, using the best estimate of the unknown ΔW_j^*. If we use the unknown from the previous iteration, this is a Jacobi iteration. Gauss-Seidel reduces the memory by storing only one solution vector. The method is *block* Gauss-Seidel because every term is a 4×4 matrix (in 2D): we need to invert a 4×4 matrix.

In practice, the Gauss-Seidel iteration will not converge because this linear system is too stiff (the matrix is not necessarily diagonally dominant). We must fix the number of iteration cycles and use the last available solution. 2 or 3 cycles of forward and backward sweeps are usually sufficient.

We normally multiply the solution update by an under-relaxation factor. Having a matrix with a low bandwidth helps: the structured mesh is optimal.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Gauss-Seidel method

This is similar to the Jacobi iteration, but we use the latest available solution values.

Assume $x^{(0)}$
while not converged **do**
  for $i = 1, 2, \ldots, n$ **do**

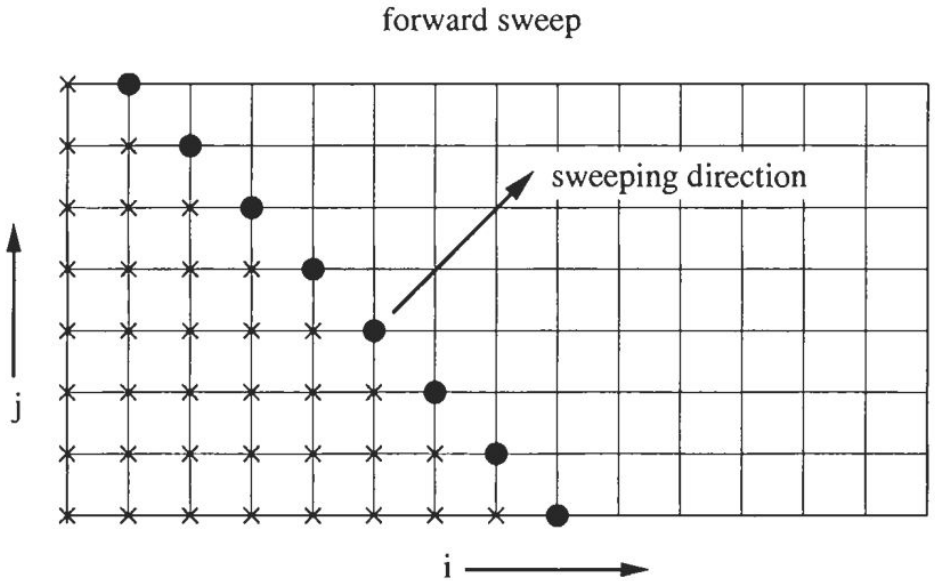
$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}\, x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}\, x_j^{(k)} \right)$$

  $k \leftarrow k + 1$

- In our case, the a_{ij} are 4×4 blocks, so we need to invert 4×4 matrices $\rightarrow$ block Gauss-Seidel.
- It is good to do forward and backward sweeps to balance the scheme.
- For structured grids, we can make this more efficient by sweeping along diagonals.

Block LU-SGS (Lower-Upper Symmetric Gauss-Seidel)

An efficient way to iterate over cells is to sweep the domain along diagonals ($i + j = \text{const}$ lines).



Lower-Upper because we apply the lower blocks at iteration $k + 1$ and the upper blocks at iteration k (forward sweep), and inversely (backward sweep).

This maximizes the use of updated values in a given sweep. Usually, we alternate forward and backward sweeps.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Alternating direction implicit method

Can be used in Homework 2

On a structured mesh, we can treat the resolution of the implicit system as three separate problems (two in 2D):

$$\left(\mathbf{D}^I + \mathbf{L}^I + \mathbf{U}^I\right)\Delta\vec{W}^{(1)} = -\vec{R}^n$$
$$\left(\mathbf{D}^J + \mathbf{L}^J + \mathbf{U}^J\right)\Delta\vec{W}^{(2)} = \Delta\vec{W}^{(1)}$$
$$\left(\mathbf{D}^K + \mathbf{L}^K + \mathbf{U}^K\right)\Delta\vec{W}^{n} = \Delta\vec{W}^{(2)}$$

$\mathbf{D}^I$, $\mathbf{L}^I$ and $\mathbf{U}^I$ are the diagonal, lower-diagonal and upper-diagonal blocks related to the derivative of the residual in the I direction.

This makes the equations block tridiagonal, and we can use the Thomas algorithm. However, there is an error associated with solving the equations this way (factorization error), which limits the stability.

Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Point implicit / point Jacobi

Can be used in Homework 2

If we take the implicit operator and neglect the off-diagonal blocks, the scheme becomes:

$$\underbrace{\boxed{\left(I\frac{\Omega_i}{\Delta t_i}+\frac{\partial R_i}{\partial W_i}\right)}}_{\textbf{D matrix}}\Delta W_i = -R_i$$

Removing $\partial R/\partial W$, we get the explicit Euler scheme.

and the cells are not coupled to one another anymore. This scheme is not unconditionally stable and does not increase the CFL limit, but it can converge faster than the explicit solver.

We can apply this as a preconditioner to the multistage scheme:

$$W^{(k+1)} = W^{(0)} - \alpha_k\,\omega\,[\mathbf{D}]^{-1}\hat{R}\left(W^{(k)}\right), \qquad k = 0,\ldots,4$$

D is a 4×4 matrix, which can be directly inverted.

Ref: Cagnone, J. S., Sermeus, K., Nadarajah, S. K., & Laurendeau, E. (2011). Implicit multigrid schemes for challenging aerodynamic simulations on block-structured grids. *Computers & Fluids*, 44(1), 314-327.

Performance of implicit schemes

In practice, the performance of implicit schemes is limited by:

1. The approximation of the fluxes used in the implicit operator. It is best to have the exact Jacobian;
2. How well we solve the linear system of equations.

In practice: the matrix is approximated and SGS is unable to converge the system $\rightarrow$ sub-optimal convergence.

Some solvers use an exact Jacobian and GMRES $\rightarrow$ this allows quadratic convergence. With $\Delta t \rightarrow \infty$, the implicit scheme becomes Newton's method.

GMRES

Can be used in Homework 2 but challenging

We write the implicit system as $A\,x = b$, with $A = I\,\frac{\Omega}{\Delta t} + \frac{\partial R}{\partial W}$, $x = \Delta W$, $b = -R$.

GMRES (Generalized Minimal RESidual) looks for the solution in the Krylov subspace built with $r_0 = b - A\,x_0$:

$$\mathcal{K}_m = \text{span}\{r_0,\ A r_0,\ A^2 r_0,\ \ldots,\ A^{m-1} r_0\}, \qquad x_m = x_0 + V_m\,y_m \ \text{ minimizes } \ \|b - A\,x\|_2$$

$\beta = \|r_0\|$, $v_1 = r_0/\beta$
for $j = 1, \ldots, m$ **do** ▷ Arnoldi
  $w = A\,v_j$
  for $i = 1, \ldots, j$ **do**
    $h_{ij} = w \cdot v_i$, $w \leftarrow w - h_{ij} v_i$
  $h_{j+1,j} = \|w\|$, $v_{j+1} = w/h_{j+1,j}$
$y_m = \arg\min_y \|\beta e_1 - \bar{H}_m\,y\|_2$
$x_m = x_0 + V_m\,y_m$

- Only needs matrix-vector products $A\,v$;
- Memory grows with m. We restart every 20 to 50 iterations;
- Needs a preconditioner (LU-SGS, ILU);
- Available in libraries like PETSc.

Ref: Saad, Y., & Schultz, M. H. (1986). GMRES: a generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7(3), 856-869.



Jacobian-Free Newton-Krylov (JFNK)

Can be used in Homework 2 but challenging

We apply Newton's method to $R(W) = 0$ (with a time step to be robust far from the solution):

$$\left(I\frac{\Omega}{\Delta t} + J\right)\Delta W = -R(W), \qquad J = \frac{\partial R}{\partial W}$$

GMRES only needs the product $J\,v$. We can approximate it with the residual:

$$J\,v \approx \frac{R(W + \varepsilon v) - R(W)}{\varepsilon}, \qquad \varepsilon \approx \frac{\sqrt{\epsilon_{\text{machine}}}\,(1 + \|W\|)}{\|v\|}$$

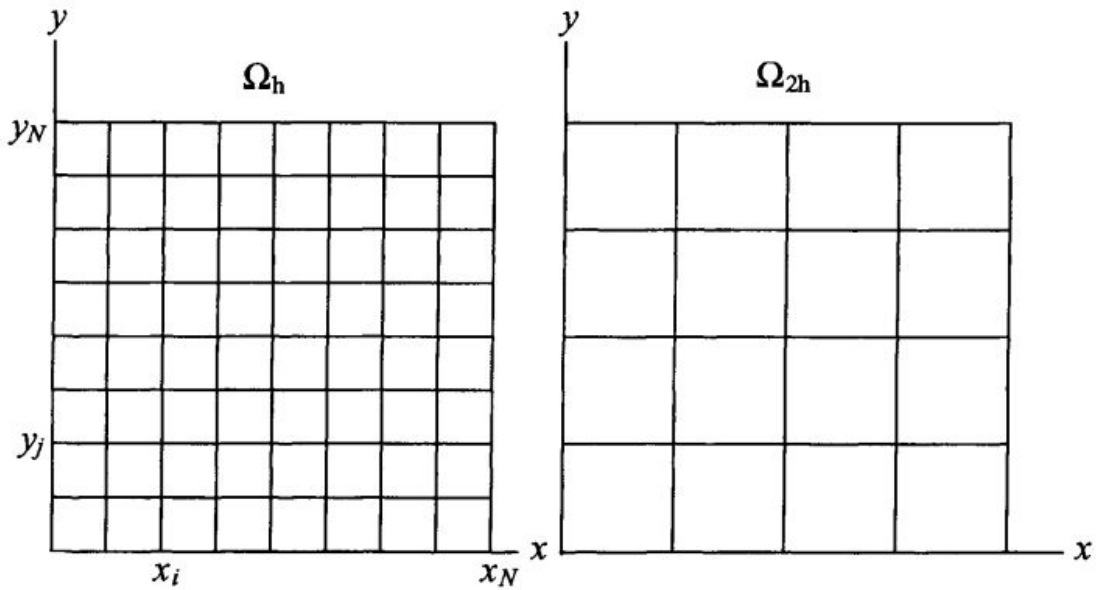
- The Jacobian matrix is neither computed nor stored, and it is the exact one;
- One residual evaluation per GMRES iteration;
- We still need a preconditioner (first-order implicit operator);
- We increase the CFL when the residual decreases: $\sigma^n = \sigma^0 \|R^0\|/\|R^n\|$. Close to the solution, the convergence is quadratic.

Ref: Knoll, D. A., & Keyes, D. E. (2004). Jacobian-free Newton-Krylov methods: a survey of approaches and applications. *Journal of Computational Physics*, 193(2), 357-397.

Multigrid

Can be used in Homework 2 but challenging

Geometric multigrid is more easily employed on structured grids, because the family of grids can be generated by merging cells in the I, J and K directions. We normally use a factor of two. Hence, the coarsest possible grid is limited by the number of times the grid dimensions can be divided by two.



Ref: Cebeci, T., Shao, J. P., Kafyeke, F., & Laurendeau, E. (2005). *Computational Fluid Dynamics for Engineers*. Horizons Publishing / Springer.

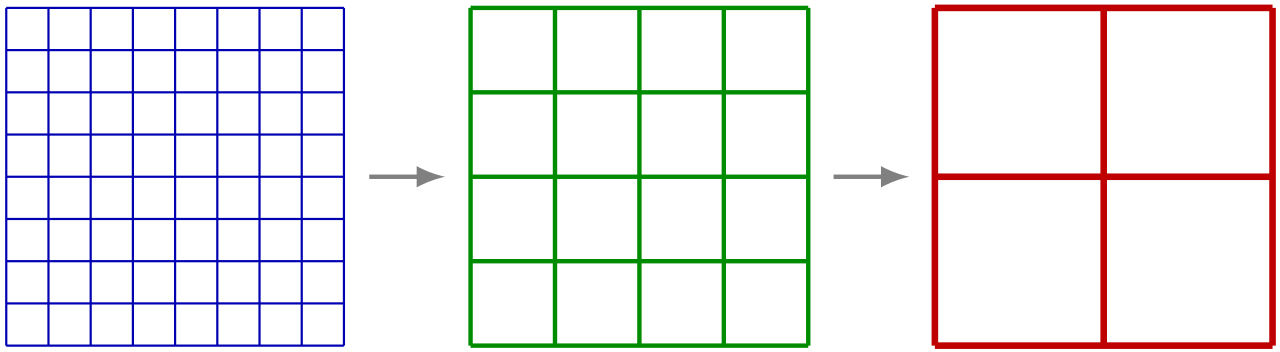
Full multigrid

Can be used in Homework 2 but challenging

A simple strategy to save time is to compute the solution on the coarse grid, and interpolate the solution on the fine grid as initial condition. This can be done on as many levels as we want.

The easiest way of doing it is to set the values of the fine cells equal to the value of the coarse cell.

This allows us to go fast on the coarse grid and to use a larger time step. It also damps the low-frequency errors more easily.



fine h (8×8) coarse $2h$ (4×4) coarser $4h$ (2×2)

Multigrid cycle

Can be used in Homework 2 but very challenging

Multigrid greatly accelerates the convergence to steady state by solving the equations on a family of fine to coarse grids (geometric multigrid). It has two effects:

- Larger time steps can be used on the coarse grids and the computational cost is smaller on coarse grids;
- Numerical schemes damp the high frequencies, the coarse grids turn the low frequencies into high frequencies which are more effectively damped.

Accuracy of the fine grid is retained.

It is also possible to do an algebraic multigrid by "coarsening" the implicit matrix. This is outside the scope of this course.

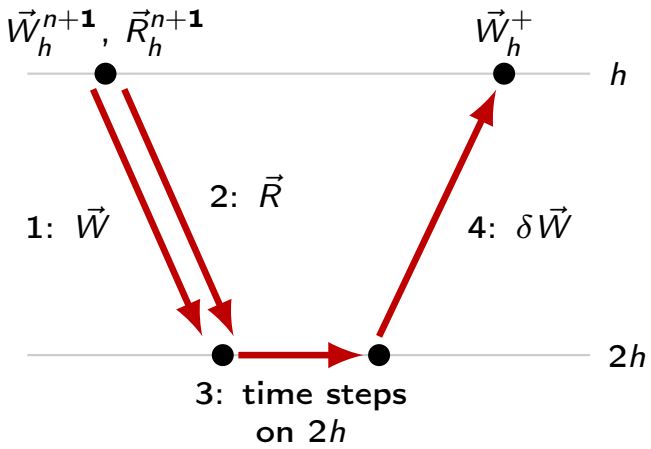
Ref: Cebeci, T., Shao, J. P., Kafyeke, F., & Laurendeau, E. (2005). *Computational Fluid Dynamics for Engineers*. Horizons Publishing / Springer.

Multigrid cycle

Can be used in Homework 2 but challenging

The basic idea of geometric multigrid is to (after a few time steps on the fine grid):

1. Transfer the fine grid solution to the coarse grid;
2. Compute a forcing function;
3. Iterate on the coarse grid;
4. Interpolate the coarse grid correction back onto the fine grid.



h: fine grid, $2h$: coarse grid

Ref: Cebeci, T., Shao, J. P., Kafyeke, F., & Laurendeau, E. (2005). *Computational Fluid Dynamics for Engineers*. Horizons Publishing / Springer.

Multigrid cycle

Can be used in Homework 2 but challenging

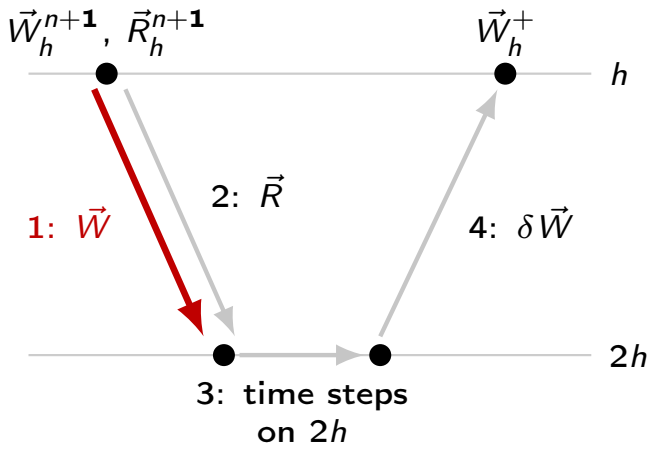
1. Transfer the fine grid solution to the coarse grid by interpolation:

$$\vec{W}_{2h}^{(0)} = \hat{I}_h^{2h}\, \vec{W}_h^{n+1}$$

$\hat{I}_h^{2h}$: interpolation operator

h: fine grid

$2h$: coarse grid



Ref: Cebeci, T., Shao, J. P., Kafyeke, F., & Laurendeau, E. (2005). *Computational Fluid Dynamics for Engineers*. Horizons Publishing / Springer.

Multigrid cycle

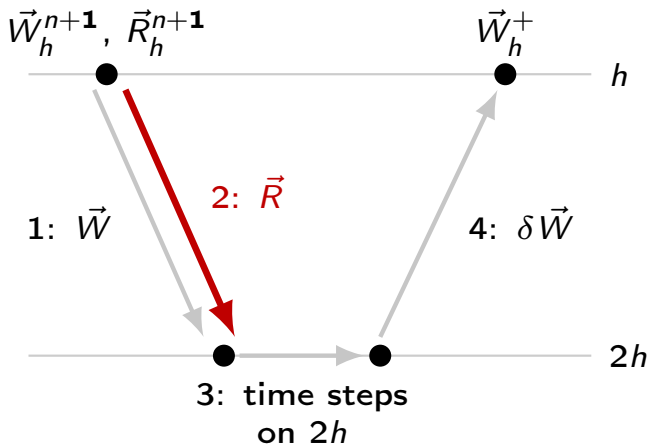
Can be used in Homework 2 but challenging

2. Compute a forcing function based on the coarse grid residual and the interpolated fine grid residual:

$$(\vec{Q}_F)_{2h} = I_h^{2h}\,\vec{R}_h^{n+1} - \vec{R}_{2h}^{(0)}$$

I_h^{2h}: restriction operator

If the fine grid is converged, the forcing function cancels the coarse grid residual and there is no correction.



Ref: Cebeci, T., Shao, J. P., Kafyeke, F., & Laurendeau, E. (2005). *Computational Fluid Dynamics for Engineers*. Horizons Publishing / Springer.

Multigrid cycle

Can be used in Homework 2 but challenging

3. Compute the new solution on the coarse grid (with the forcing function as a source term):

$$(\vec{R}_F)_{2h} = \vec{R}_{2h} + (\vec{Q}_F)_{2h}$$

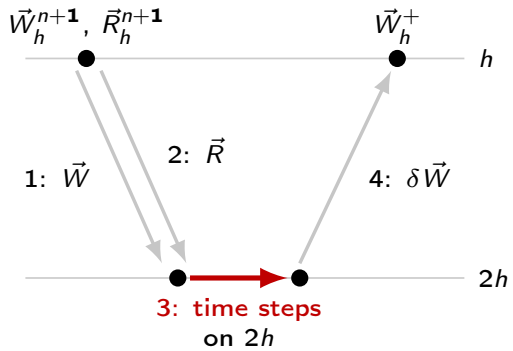
$$\frac{\mathrm{d}}{\mathrm{d}t}\vec{W}_{2h} = -\frac{1}{\Omega_{2h}}(\vec{R}_F)_{2h} = -\frac{1}{\Omega_{2h}}\left[\vec{R}_{2h} + (\vec{Q}_F)_{2h}\right]$$

Same time integration as on the fine grid. If explicit multistage scheme:

$$\vec{W}_{2h}^{(k)} = \vec{W}_{2h}^{(0)} - \alpha_k \frac{\Delta t_{2h}}{\Omega_{2h}}\left[\vec{R}_{2h}^{(k-1)} + (\vec{Q}_F)_{2h}\right], \quad k = 1, \ldots, m$$

$$\vec{W}_{2h}^{n+1} = \vec{W}_{2h}^{(m)}$$

Any temporal scheme and acceleration method can be used.



Ref: Cebeci, T., Shao, J. P., Kafyeke, F., & Laurendeau, E. (2005). *Computational Fluid Dynamics for Engineers*. Horizons Publishing / Springer.

Multigrid cycle

Can be used in Homework 2 but challenging

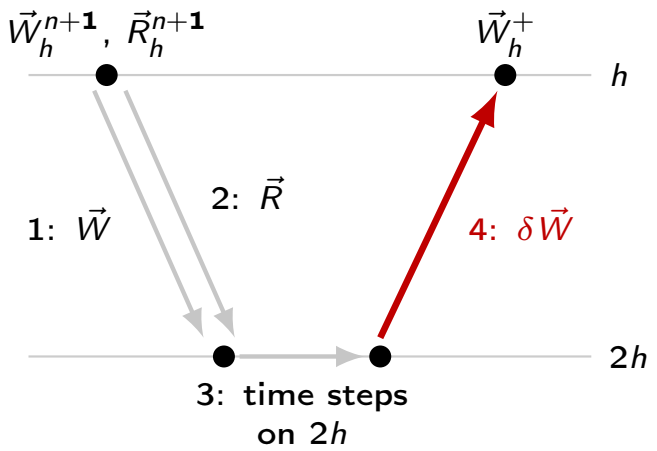
4. Interpolate (prolongation) the coarse grid correction back onto the fine grid:

$$\delta \vec{W}_{2h} = \vec{W}_{2h}^{n+1} - \vec{W}_{2h}^{(0)}$$

$$\vec{W}_h^+ = \vec{W}_h^{n+1} + I_{2h}^h \, \delta \vec{W}_{2h}$$

I_{2h}^h: prolongation operator

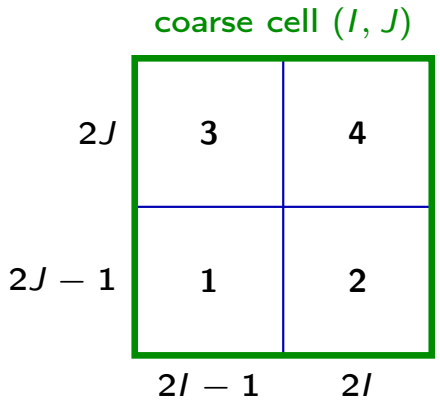
We transfer the correction, not the coarse grid solution.



Ref: Cebeci, T., Shao, J. P., Kafyeke, F., & Laurendeau, E. (2005). *Computational Fluid Dynamics for Engineers*. Horizons Publishing / Springer.

Multigrid operators

Can be used in Homework 2 but challenging



Beware of indices: the coarse cell (I, J) contains the fine cells $2I-1$, $2I$ and $2J-1$, $2J$.

Interpolation (volume-weighted average):

$$\left(\vec{W}_{2h}^{(0)}\right)_{I,J} = \frac{\sum_{k=1}^{4} \Omega_k \left(\vec{W}_h^{n+1}\right)_k}{\sum_{k=1}^{4} \Omega_k}$$

Restriction:

$$\left(I_h^{2h} \vec{R}_h^{n+1}\right)_{I,J} = \sum_{k=1}^{4} \left(\vec{R}_h^{n+1}\right)_k$$

Prolongation (simpler to implement):

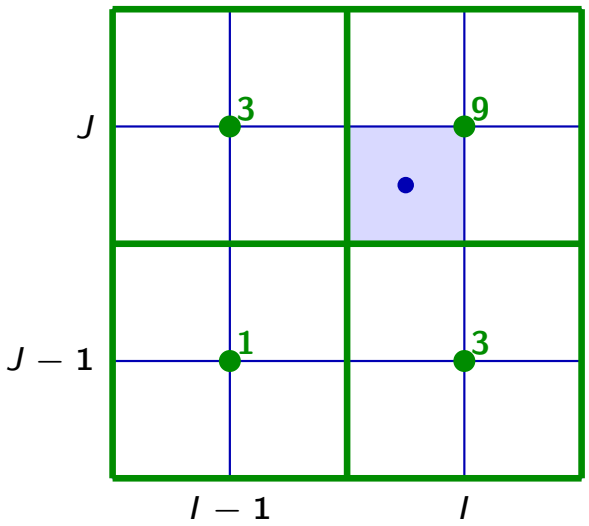
$$\left(\vec{W}_h^{+}\right)_k = \left(\vec{W}_h^{n+1}\right)_k + \left(\delta \vec{W}_{2h}\right)_{I,J}, \qquad k = 1, \ldots, 4$$

Ref: Cebeci, T., Shao, J. P., Kafyeke, F., & Laurendeau, E. (2005). *Computational Fluid Dynamics for Engineers*. Horizons Publishing / Springer.

Multigrid operators

Can be used in Homework 2 but challenging

We can also interpolate the correction from centre to node on the coarse grid, and then to the centre of the fine grid.



Fine cell 1 (shaded) and the weights of the coarse cells.

For the fine cell 1, $(2I-1, 2J-1)$:

$$\left(I_{2h}^{h}\delta\vec{W}_{2h}\right)_1 = \frac{1}{16}\left[9\left(\delta\vec{W}_{2h}\right)_{I,J} + 3\left(\delta\vec{W}_{2h}\right)_{I-1,J} + 3\left(\delta\vec{W}_{2h}\right)_{I,J-1} + \left(\delta\vec{W}_{2h}\right)_{I-1,J-1}\right]$$

The same is done for the other fine cells, with their closest coarse cells.

Ref: Cebeci, T., Shao, J. P., Kafyeke, F., & Laurendeau, E. (2005). *Computational Fluid Dynamics for Engineers*. Horizons Publishing / Springer.

Multi-level multigrid

Can be used in Homework 2 but challenging

The simple multigrid algorithm above applies to two grid levels. We can have more than two levels. For example, on the second coarse level:

$$(\vec{Q}_F)_{4h} = I_{2h}^{4h}(\vec{R}_F)_{2h}^{n+1} - \vec{R}_{4h}^{(0)} = I_{2h}^{4h}\left[\vec{R}_{2h}^{n+1} + (\vec{Q}_F)_{2h}\right] - \vec{R}_{4h}^{(0)}$$

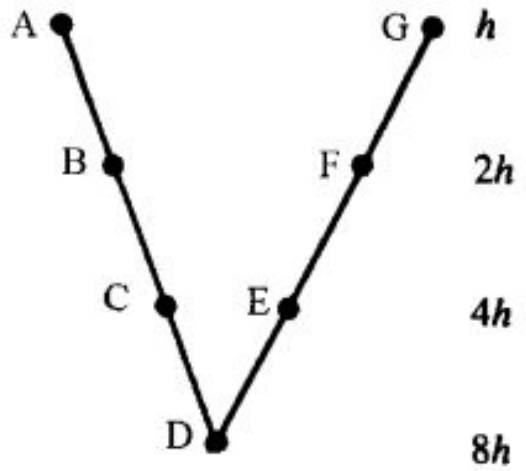
It should be noted that multiple time steps can be executed on the coarse grids. This results in a variety of multigrid strategies. Depending on how we navigate between grid levels, the strategy can be a V-cycle, a W-cycle or an F-cycle.

Ref: Cebeci, T., Shao, J. P., Kafyeke, F., & Laurendeau, E. (2005). *Computational Fluid Dynamics for Engineers*. Horizons Publishing / Springer.

Multigrid cycle types

Can be used in Homework 2 but challenging

V-cycle



W-cycle



Going down: restriction (to a coarser level). Going up: prolongation (to a finer level).

In one cycle, we use a given number of time iterations before the restriction (moving to a coarser level) and before the prolongation (moving to a finer level). This results in a very large number of potential strategies, which are calibrated for a specific algorithm.

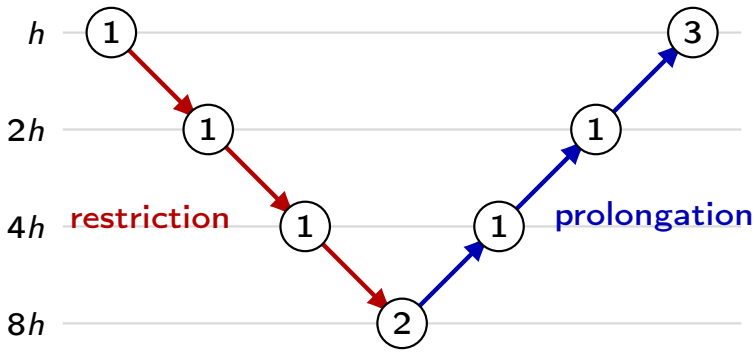
Ref: Cebeci, T., Shao, J. P., Kafyeke, F., & Laurendeau, E. (2005). *Computational Fluid Dynamics for Engineers*. Horizons Publishing / Springer.

Multigrid cycle types

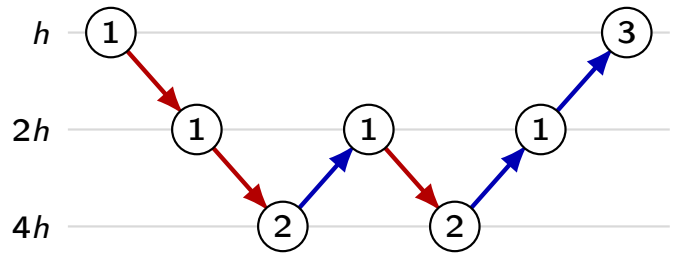
Can be used in Homework 2 but challenging

The numbers in the circles are the numbers of iterations on each level.

V-cycle: 1-1-1-2-1-1-3



W-cycle: 1-1-2-1-2-1-3



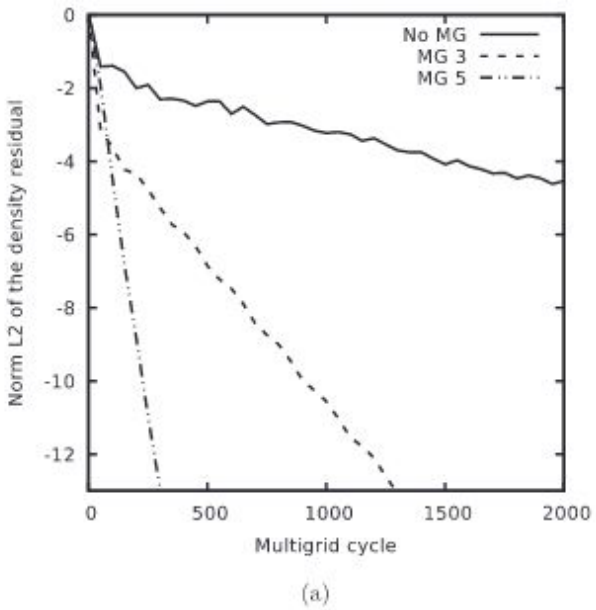
- The W-cycle visits the coarse grids more often;
- In 2D, a coarse grid has 4 times fewer cells. The time cost of the V-cycle is about $1 + \frac{1}{4} + \frac{1}{16} + \frac{2}{64} + \frac{1}{16} + \frac{1}{4} + 3 \approx 4.7$ fine grid iterations.

Ref: Loppi, N. A., Witherden, F. D., Jameson, A., & Vincent, P. E. (2018). A high-order cross-platform incompressible Navier-Stokes solver via artificial compressibility with application to a turbulent jet. *Computer Physics Communications*, 233, 193-205.

Multigrid

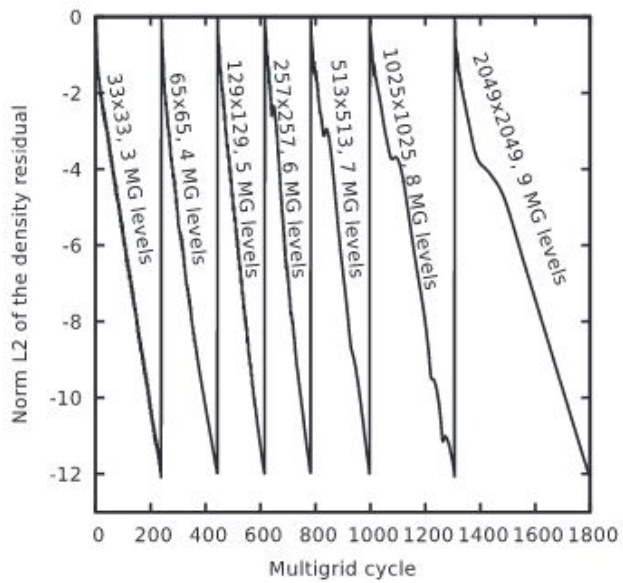
Can be used in Homework 2 but challenging

For elliptic equations, it can be demonstrated that the convergence rate on the fine grid is the one on the coarse grid. The Euler equations are hyperbolic and such a demonstration cannot be done. However, in practice the convergence rate approaches that of the coarse grid.



(a)

257×257 mesh



Source: Plante, F., & Laurendeau, É. (2018). Acceleration of Euler and RANS solvers via selective frequency damping. *Computers & Fluids*, 166, 46-56.

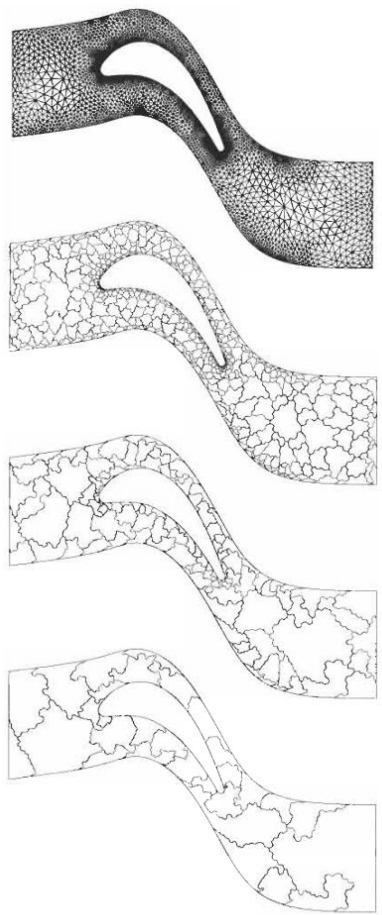
Multigrid for unstructured grids

Not used in this course

Implementing multigrid is more complex on unstructured grids:

- Generating the coarse grids is more challenging;
- The interpolation operators are more complex.

Multigrid is also less efficient on unstructured grids than structured ones.

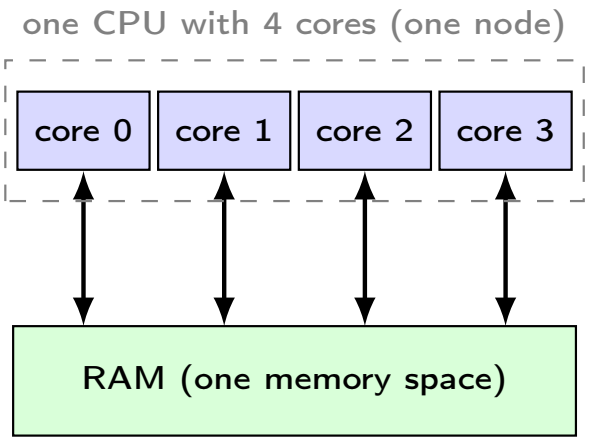


Ref: Blazek, J. (2015). *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann.

Parallelization

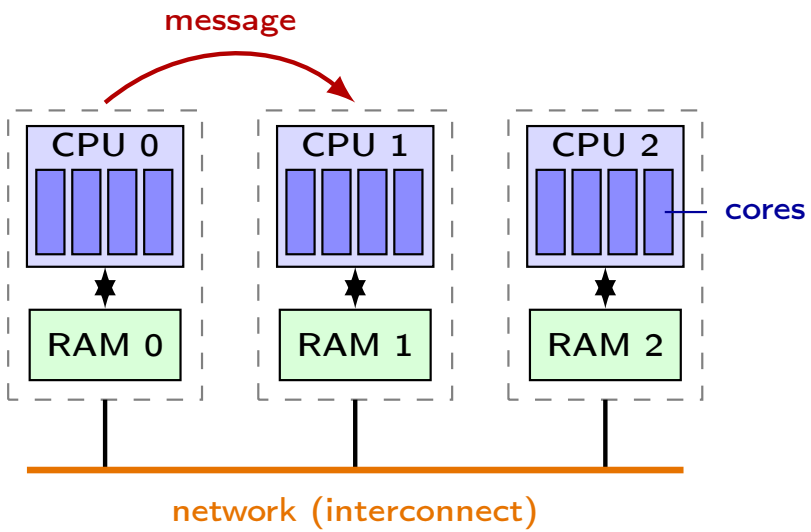
Shared memory and distributed memory

Shared memory (OpenMP)



all threads read and write
the same arrays

Distributed memory (MPI)



each process has its own memory;
data is exchanged by messages

All the cores see the same memory. This is like your typical Desktop

Each process has its own memory. We need to send messages between the nodes. This is how supercomputers are constructed.

We call a node a combination of one CPU (with multiple cores) connected to one RAM. We can also combine both: MPI between the nodes and OpenMP inside a node.

Parallelization

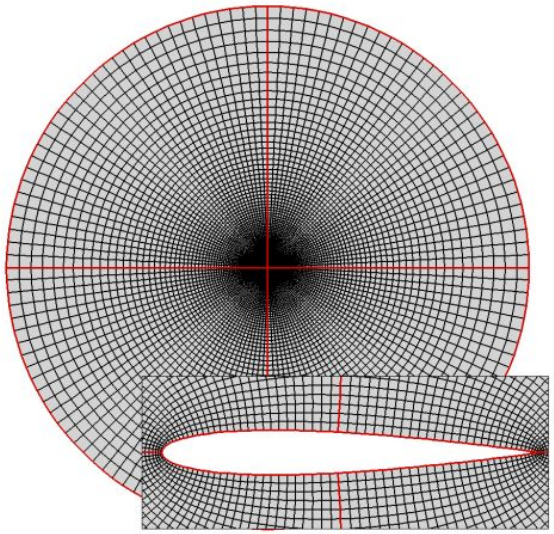
High Performance Computing is a topic closely related to Computational Fluid Dynamics. There are two main types of parallelization:

- Shared memory (OpenMP standard)
  - All memory is on the same RAM and thus accessible from all CPU cores;
  - Simple to implement on an existing serial code;
  - Load balancing needs to be optimized to achieve high performance;
  - Limits the number of CPU cores and the RAM which can be used (one compute node).
- Distributed memory (Message Passing Interface, MPI)
  - Memory is distributed on several nodes (combination of a RAM and a CPU);
  - We run one process per CPU core (`if` statements can control which part of the code is done by a core);
  - For CFD, this means splitting and distributing mesh blocks;
  - Requires coding the communication between the RAMs, making it more complex;
  - Communication needs to be optimized to achieve high performance;
  - Allows harnessing a very large number of CPU cores on modern supercomputers.

Parallelization is a complex topic and more options exist (Chapel language, GPU programming, etc.).

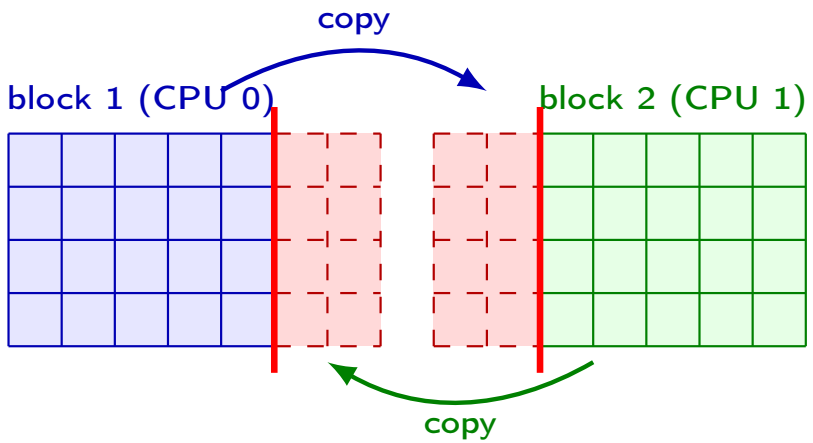


Multiblock mesh



Red lines are block boundaries

For distributed memory, we distribute the blocks among the CPUs. Each block needs the values of its neighbours at the interfaces.



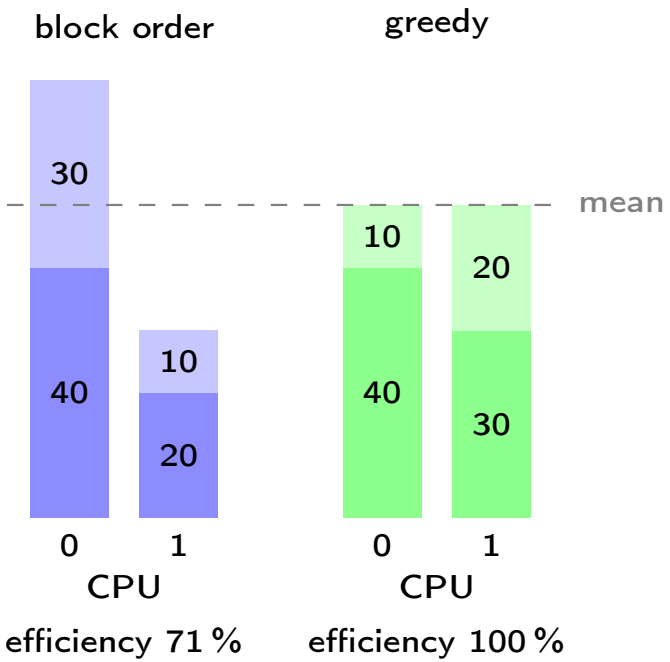
2 layers of ghost cells (red) on each side of the interface, filled with the interior values of the neighbour block

- We need a connectivity table: for each block face, the neighbour block and face (e.g. $i_{\max}$ of block 1 is connected to $i_{\min}$ of block 2);
- Ghost cells are exchanged at each stage;
- If the two blocks are on the same node, it is a simple copy.

Load balancing

The CPUs wait for each other at each exchange of ghost cells. The time of one iteration is given by the most loaded CPU.

$$\text{load balance efficiency} = \frac{\text{mean load}}{\text{maximum load}}$$



Four blocks of 40, 30, 20 and 10 thousand cells on 2 CPUs.

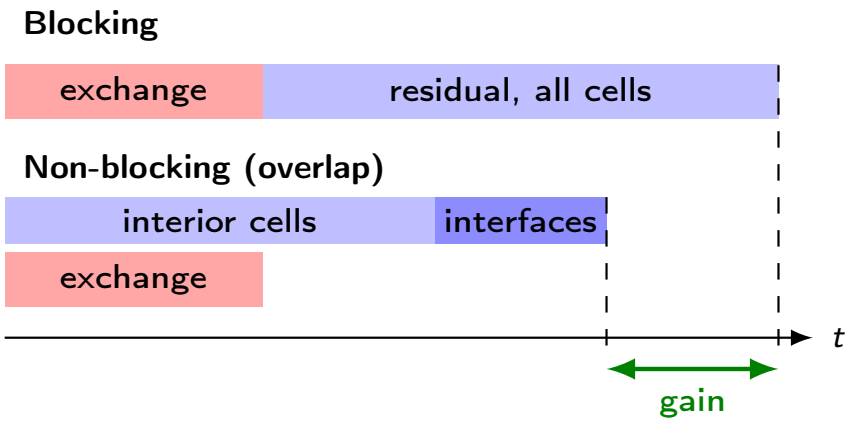
On structured grid

- Simple algorithm: sort the blocks from largest to smallest and give each one to the least loaded CPU;
- Split the large blocks if needed (keep them divisible by 2 for multigrid);
- Put neighbour blocks on the same CPU to reduce communication.

On unstructured grid we can split the mesh evenly, but we need to be carefull to minimize the interfaces to limit the communication time.

Optimization of the MPI exchange

Time to send one message: $t \approx \alpha + \frac{\text{size}}{\beta}$, latency $\alpha \sim 1\,\mu$s, bandwidth $\beta \sim 10$ GB/s.
It is better to send a few large messages than many small ones.



The exchange is done while computing the interior cells.

post `MPI_Irecv` (each neighbour)
pack the ghost data, `MPI_Isend`
residual of the interior cells
`MPI_Waitall`, unpack the ghost cells
residual of the cells near the interfaces

Non-blocking communication (`MPI_Isend`/`MPI_Irecv`) also avoids deadlocks.

Optimization of the MPI exchange

- Send one message per neighbour CPU, with all the variables and the 2 layers of ghost cells;
- Exchange only what is needed ($\vec{W}$). The metrics are exchanged once at the start;
- Reduce the size of the interfaces between CPUs;
- Global operations (`MPI_Allreduce` for the residual norm or the forces) synchronize all the CPUs. Do them only every N iterations;
- On the coarse multigrid levels, the messages are small and the latency dominates;
- Measure the communication time separately (`MPI_Wtime`).

Parallel efficiency - Amdahl's law

Amdahl's law is the relation between the theoretical speedup of a parallel code and the fraction of the code which is parallel. With s cores:

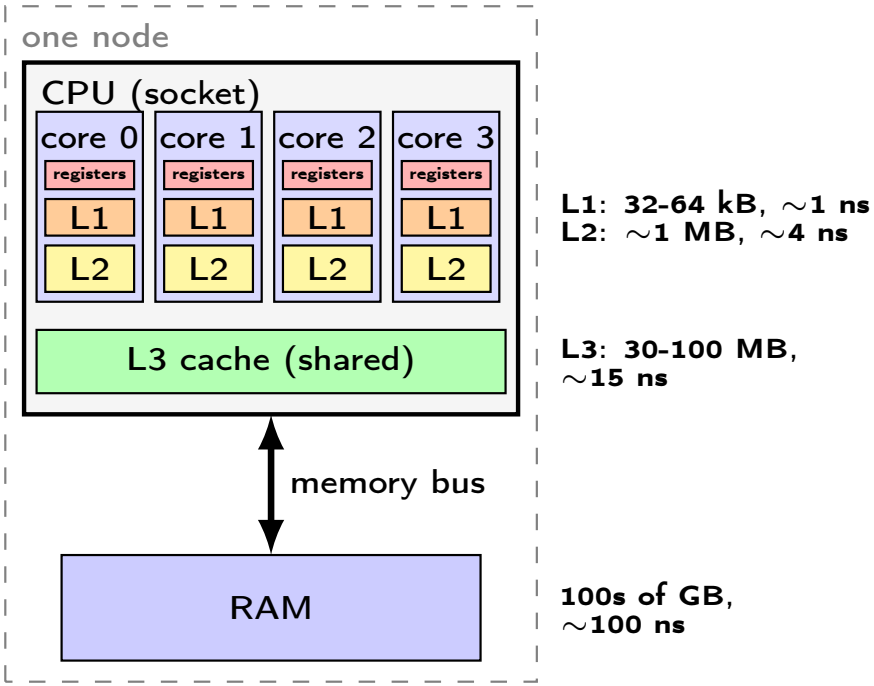
$$S_{\text{latency}}(s) = \frac{1}{(1-p) + \frac{p}{s}} \xrightarrow[s\to\infty]{} \frac{1}{1-p}$$

Let's imagine a code takes 10 hours to run and 9 hours are spent in parts which can be parallelized, then $p = 0.9$. The maximum speedup S is 10 and the code cannot take less than an hour to run.

This is a law of diminishing returns. To have high parallel efficiency, $1 - p$ must be very small. CFD codes run on hundreds to thousands of CPU cores and a small serial part can seriously degrade performance.

Sometimes, the speedup is better than perfect because the code becomes more efficient with a smaller load per core.

Superlinear efficiency



Typical orders of magnitude (size, access time).

The data come from the RAM through the caches. Reading the RAM is about 100 times slower than reading the L1 cache.

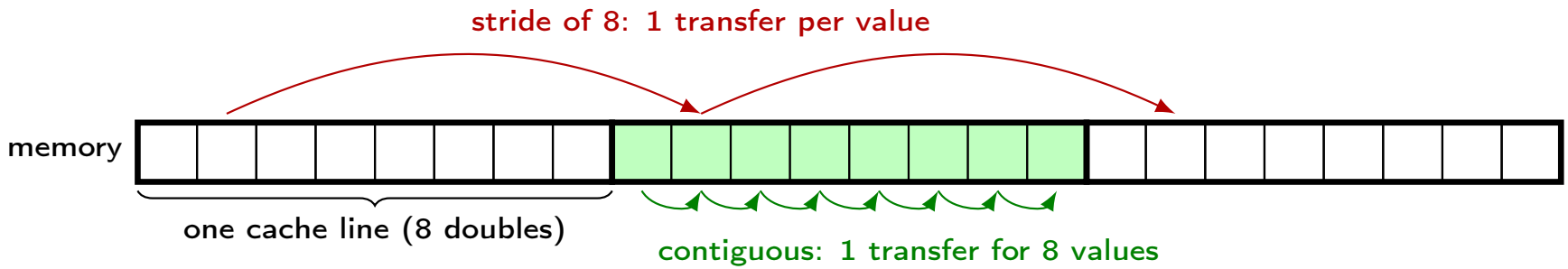
In strong scaling, each core has a smaller part of the mesh when we add cores:

- 1 core: 10^6 cells, a few hundred MB $\rightarrow$ in the RAM;
- 64 cores: a few MB per core $\rightarrow$ in the cache.

The cores wait less for the memory and the speedup can be greater than the number of cores (superlinear speedup).

Loop order and cache

The memory is transferred to the cache by lines of 64 bytes (8 doubles). When we read a value, the next 7 values are also in the cache.



Fast

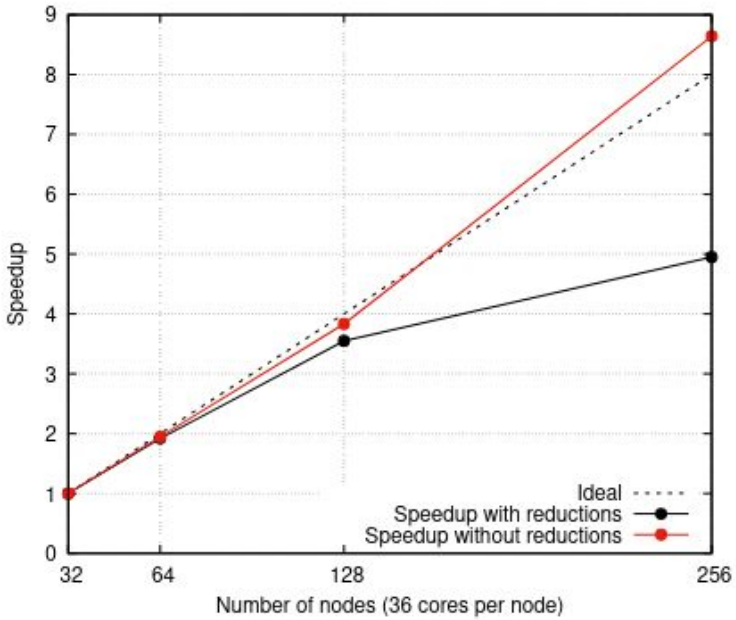
```
for (int j = 0; j < nj; ++j)
  for (int i = 0; i < ni; ++i)
    R[j][i] = F[j][i+1] - F[j][i];
```

Slow

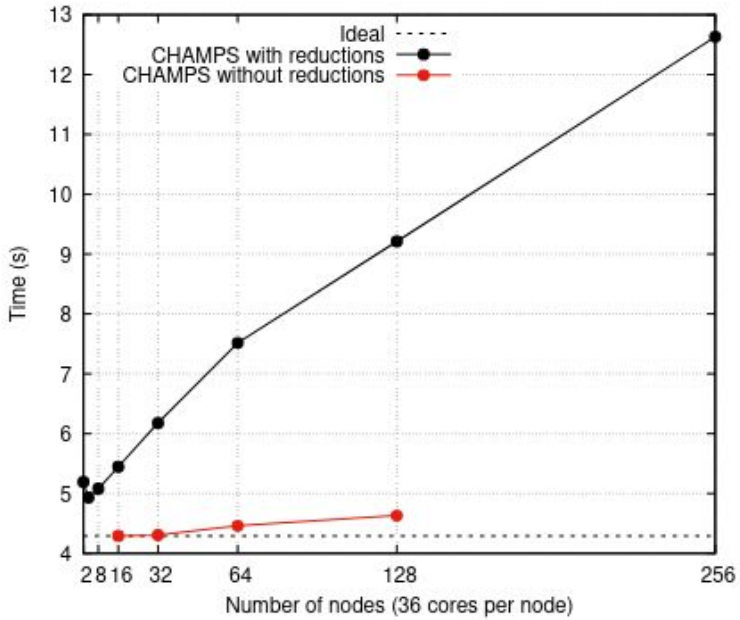
```
for (int i = 0; i < ni; ++i)
  for (int j = 0; j < nj; ++j)
    R[j][i] = F[j][i+1] - F[j][i];
```

- The inner loop must be on the index which is contiguous in memory (last index in C/C++, first index in Fortran);
- This also allows the compiler to vectorize the loop;
- False sharing can minimize software performance. Cores can be blocked by the need to access a same cache line.

Parallel efficiency - strong and weak scaling



(a) Strong scaling



(b) Weak scaling

Strong scaling: same problem size, we increase the number of CPU cores. Weak scaling: we increase the size of the problem with the number of CPU cores. Ideally, the time should be the same.

Ref: Parenteau, M., Bourgault-Côté, S., Plante, F., Kayraklioglu, E., & Laurendeau, E. (2021). Development of parallel CFD applications with the Chapel programming language. In *AIAA Scitech 2021 Forum* (p. 0749).

Practical application

Choosing the proper number of cores to run a program is important:

- To reduce wait time. Supercomputer usually use a queuing system to distribute the ressources;
- To reduce the cost of the simulation. Using more CPU at a lower efficiency cost more money;
- Run more simulation over the same time. It is more efficient to run more simulations with a lower number of CPUs on each;
- Minimize the licence cost. Some software requires one licence per CPU.

OpenMP C++ example

OpenMP is an interface to do shared-memory parallel computing with the C, C++ and Fortran languages.

You can add OpenMP directives to your code. They are handled by the compiler to make your software use more than one thread (CPU core).

Compile with `g++ -fopenmp`.

```
1  #include <cstdio>
2  #include <time.h>
3  #include <omp.h>
4  int main()
5  {
6      double start;
7      double end;
8      int iTot = 0;
9      start = omp_get_wtime();
10     #pragma omp parallel
11     {
12         int tid=omp_get_thread_num();
13         printf("Hello world from omp thread %d\n", tid);
14         #pragma omp for schedule(static)
15         for(int n=0; n<100; ++n)
16         {
17
18             int i = 0;
19             int jmax = 2000000;
20             for(int j=0; j<jmax; ++j)
21             {
22                 i+=1;
23             }
24             #pragma omp critical
25             {
26                 iTot+=1;
27             }
28         }
29     }
30     printf("iTot = %d\n", iTot);
31     end = omp_get_wtime();
32     printf("Work took %f seconds\n", end - start);
33 }
```

OpenMP C++ example

Line by line (see the previous slide):

3 `#include <omp.h>`: include for OpenMP

9 `omp_get_wtime()`: get the wall-clock time

10 `#pragma omp parallel`: start a structured block that runs on multiple threads

12 `omp_get_thread_num()`: get the thread number

14 `#pragma omp for schedule(static)`: inform the compiler that this loop is to be distributed among the threads

24 `#pragma omp critical`: inform the compiler that this block must be executed by a single thread at a time

Parallel implementation

Can be used in Homework 2 but challenging

If you want to make use of some parallelism:

- Start by debugging without parallelism;
- Add OpenMP directives to a limited number of loops;
- Verify regularly that the convergence is exactly the same;
- Beware of race conditions (if the iterations of a loop depend on each other, you may get random behaviour if the parallelism is not done right);
- Beware of variables incremented in the loop (use `omp critical` or a `reduction` clause).

In Linux, we set an environment variable to define the number of threads:
`export OMP_NUM_THREADS=8`

Performance optimization

Optimizing the performance of a code can also be done by:

- Using optimization flags in the compilation (`-O2` or `-O3`);
- Profiling the code run time:
  - Find the most costly routines and optimize them;
  - Requires removing compiler optimization;
  - Beware: some gains identified this way are already handled by the compiler optimization (like a constant `if` in a loop);
- Optimizing the software to run on specific hardware:
  - Split or merge loops to optimize cache memory access.

Ref: Sermeus, K., Laurendeau, E., & Parpia, F. (2007). Parallelization and performance optimization of Bombardier multiblock structured Navier-Stokes solver on IBM eServer Cluster 1600. In *45th AIAA Aerospace Sciences Meeting and Exhibit* (p. 1109). (on Moodle)

How to proceed in the code development?

The simplest solver should be a centred scheme with artificial dissipation (or a Roe scheme) with an explicit Euler time integration. Then:

- Implementing a multistage scheme is required for the homework;
- Adding local time stepping is straightforward (only needs an array of time steps);
- Adding the residual smoothing allows to increase the CFL number and to double the convergence rate.

Advanced features:

- Adding a point-implicit preconditioner allows us to test the diagonal blocks of the implicit matrix (can be a stepping stone towards an implicit solver);
- An implicit solver with LU-SGS is quite challenging;
- Multigrid is the most efficient acceleration, but very challenging.

